BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 2 Practical No: 9

Roll no: 57 PRN No:12410559 Name of Student: Avni Selote

Subject Name : Deep Learning

Problem Statement : Image Classification using Pretrained Vision Transformer and CNN

Objective: Perform image classification on CIFAR-10 using (a) a conventional Convolutional Neural Network (CNN) trained from scratch and (b) a pretrained Vision Transformer (ViT) fine-tuned on CIFAR-10, and then compare the two models on accuracy, F1-score, training time and model size.

Item	Details
Dataset	CIFAR-10 (tf.keras.datasets.cifar10) – 60,000 colour images (32×32×3), 10 classes
Model 1	A 3-block CNN (Conv + BatchNorm + ReLU + MaxPool) trained from scratch
Model 2	ViT-B/16 (224×224) pretrained on ImageNet, loaded from KerasHub, with a new 10-class head, trained by two-stage transfer learning

Dataset :

CIFAR-10 contains 60,000 colour images of size 32×32×3 in 10 classes (6,000 per class): 50,000 for training and 10,000 for testing. The classes are mutually exclusive (airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck). The images are tiny and low resolution, which makes the dataset a good small benchmark.

Data Preprocessing

CNN: works directly on the original 32×32×3 images. Pixel values are scaled from [0, 255] to [0, 1] with a Rescaling layer inside the model.
Pretrained ViT: the chosen model (ViT-B/16) was pretrained on 224×224 ImageNet images. Its position embeddings are learned for a 14×14 grid of 16×16 patches (196 patches). A 32×32 image would give only 4 patches, and the pretrained position embeddings would no longer match. So we resize CIFAR-10 images to 224×224 and normalise them exactly as the pretrained model expects (KerasHub's preset normalisation values are read in Section 11).

Efficient pipeline (memory decision):

- Data is stored once as small uint8 arrays. Storing 50,000 resized 224×224×3 float images would need ~30 GB of RAM, which is impossible on Colab.
- So the tf.data pipeline feeds the original 32×32 images, and the resize to 224×224 happens inside the ViT model, on the GPU, one batch at a time.
- shuffle (training only) randomises the order every epoch, batch groups images for the GPU, and prefetch prepares the next batch while the GPU works on the current one.
- We do not use cache() for the pipeline because the arrays are already in RAM, and caching resized images would be far too large.

Pretrained Vision Transformer

A Vision Transformer (ViT) splits an image into fixed-size patches (here 16×16), flattens each patch and projects it linearly into an embedding vector (a "token"). A learnable [CLS] token and position embeddings are added and the token sequence is processed by a stack of Transformer encoder blocks. Each block uses multi-head self-attention (every patch can look at every other patch) and an MLP. The final [CLS] vector is a summary of the whole image used for classification.

Property	**Value**
Model	ViT-B/16 (Base size, 16×16 patches): 12 encoder blocks, 12 heads, hidden size 768, ~86M parameters
Source	KerasHub preset vit_base_patch16_224_imagenet (keras_hub.models.ViTBackbone.from_preset)
Pretraining	Yes, ImageNet (supervised image classification on ImageNet; see the exact preset description printed below)
Expected input	224 × 224 × 3 images (14 × 14 = 196 patches + 1 [CLS] token = 197 tokens)

Comparison

- Accuracy: the CNN scored 80.49% and the pretrained ViT 97.76% on the 10,000 test images; the pretrained ViT was better by 17.27 percentage points.
- F1-score (macro): CNN 0.8032 vs ViT 0.9776. Because CIFAR-10 is balanced, F1 follows accuracy closely.
- Test loss: CNN 0.5648 vs ViT 0.0700 (lower means more confident correct predictions).
- Training time: CNN 3.69 min (20 epochs) vs ViT 39.51 min (5 epochs). The CNN was 10.7x faster in total.
- Time per epoch: CNN 11.1 s vs ViT 474.2 s.
- Parameters: CNN 323,946 vs ViT 85,806,346 (ViT is about 265x larger).

- Generalization: the gap between best validation accuracy and test accuracy is 0.57 pp for the CNN and 0.18 pp for the ViT; small gaps mean the validation set was a reliable estimate of test performance.
- Effect of pretraining: the ViT reached its result with only 5 epochs, while the CNN needed up to 20 epochs from random initialisation. This supports the idea that pretrained representations give a large head start on a small, low-resolution dataset.

Coclusion

1. Better test accuracy: the pretrained ViT (97.76% vs 80.49%).
2. Faster training: the CNN (3.69 min vs 39.51 min).
3. More parameters: the pretrained ViT (85,806,346 vs 323,946).
4. Advantage of pretraining: the ViT starts with general visual features learned on ImageNet, so only a new head and the top 4 blocks were trained for 5 epochs, instead of learning everything from random weights.
5. Main differences: a CNN uses local convolution filters with weight sharing (strong image bias, few parameters); a ViT splits the image into patches and uses global self-attention (more flexible, needs large-scale pretraining, more computation and a larger input size).
6. Model I would choose for this CIFAR-10 task: the pretrained ViT. The 17.27-point accuracy gain justifies the extra training time and model size when accuracy is the main goal.

Code and output :

```python
import os
os.environ["KERAS_BACKEND"] = "tensorflow"   # must be set BEFORE importing keras: we want the TensorFlow backend

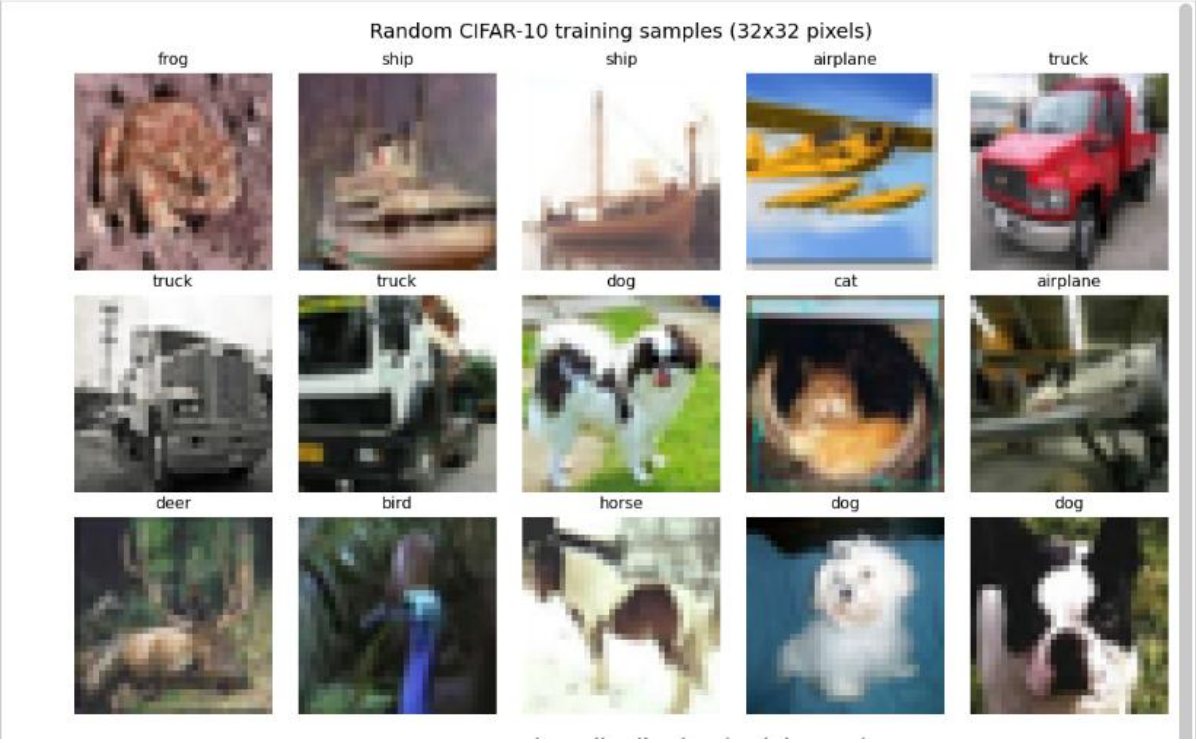
import random, time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

import tensorflow as tf
import keras
from keras import layers
import keras_hub                              # pretrained Vision Transformer lives here

from sklearn.metrics import classification_report, confusion_matrix, f1_score
from IPython.display import display, Markdown

print("TensorFlow :", tf.__version__)
print("Keras      :", keras.__version__, "| backend:", keras.backend.backend())
print("KerasHub   :", keras_hub.__version__)
```

```
TensorFlow : 2.20.0
Keras      : 3.13.2 | backend: tensorflow
KerasHub   : 0.26.0
```



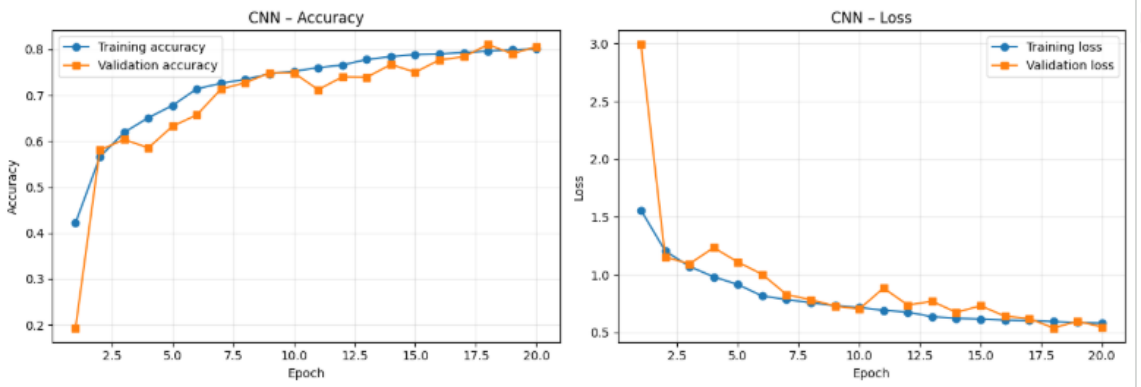

```python
def build_augmentation(name):
    # dtype="float32" keeps these layers in full precision even when mixed precision is enabled later.
    return keras.Sequential([
        layers.RandomFlip("horizontal", dtype="float32"),   # a flipped cat is still a cat (vertical flips
        layers.RandomRotation(0.03, dtype="float32"),       # about +/- 11 degrees
        layers.RandomZoom(0.1, dtype="float32"),            # zoom in/out by up to 10%
    ], name=name)

# Visualise: one image augmented 5 times (training=True switches the random behaviour ON)
demo_aug = build_augmentation("demo_aug")
img = x_train[0:1].astype("float32")
fig, axes = plt.subplots(1, 6, figsize=(13, 2.6))
axes[0].imshow(x_train[0]); axes[0].set_title("Original")
for k in range(1, 6):
    axes[k].imshow(np.clip(demo_aug(img, training=True)[0].numpy(), 0, 255).astype("uint8"))
    axes[k].set_title(f"Augmented {k}")
for a in axes: a.axis("off")
plt.show()
```



Model: "cnn_cifar10"

Layer (type)	Output Shape	Param #
cifar_image (InputLayer)	(None, 32, 32, 3)	0
cnn_augmentation (Sequential)	(None, 32, 32, 3)	0
rescale_0_1 (Rescaling)	(None, 32, 32, 3)	0
conv2d (Conv2D)	(None, 32, 32, 32)	864
batch_normalization (BatchNormalization)	(None, 32, 32, 32)	128
re_lu (ReLU)	(None, 32, 32, 32)	0
conv2d_1 (Conv2D)	(None, 32, 32, 32)	9,216
batch_normalization_1 (BatchNormalization)	(None, 32, 32, 32)	128
re_lu_1 (ReLU)	(None, 32, 32, 32)	0
max_pooling2d (MaxPooling2D)	(None, 16, 16, 32)	0
dropout (Dropout)	(None, 16, 16, 32)	0
conv2d_2 (Conv2D)	(None, 16, 16, 64)	18,432
batch_normalization_2 (BatchNormalization)	(None, 16, 16, 64)	256
re_lu_2 (ReLU)	(None, 16, 16, 64)	0
conv2d_3 (Conv2D)	(None, 16, 16, 64)	36,864
batch_normalization_3 (BatchNormalization)	(None, 16, 16, 64)	256
re_lu_3 (ReLU)	(None, 16, 16, 64)	0
max_pooling2d_1 (MaxPooling2D)	(None, 8, 8, 64)	0
dropout_1 (Dropout)	(None, 8, 8, 64)	0
conv2d_4 (Conv2D)	(None, 8, 8, 128)	73,728
batch_normalization_4 (BatchNormalization)	(None, 8, 8, 128)	512
re_lu_4 (ReLU)	(None, 8, 8, 128)	0
conv2d_5 (Conv2D)	(None, 8, 8, 128)	147,456
batch_normalization_5 (BatchNormalization)	(None, 8, 8, 128)	512
re_lu_5 (ReLU)	(None, 8, 8, 128)	0
max_pooling2d_2 (MaxPooling2D)	(None, 4, 4, 128)	0



```
CNN Test loss     : 0.5648
CNN Test accuracy : 0.8049  (80.49%)
CNN Test macro-F1 : 0.8032

Classification report (CNN):

              precision    recall  f1-score   support

    airplane     0.8266    0.8200    0.8233      1000
  automobile     0.9262    0.9030    0.9144      1000
        bird     0.7683    0.6830    0.7231      1000
         cat     0.6878    0.6190    0.6516      1000
        deer     0.7626    0.7870    0.7746      1000
         dog     0.7778    0.6720    0.7210      1000
        frog     0.7026    0.9120    0.7937      1000
       horse     0.8669    0.8340    0.8502      1000
        ship     0.8814    0.9070    0.8940      1000
       truck     0.8612    0.9120    0.8859      1000

    accuracy                         0.8049     10000
   macro avg     0.8061    0.8049    0.8032     10000
weighted avg     0.8061    0.8049    0.8032     10000
```

```python
def build_vit_classifier(backbone):
    inputs = keras.Input(shape=(32, 32, 3), name="cifar_image")
    x = build_augmentation("vit_augmentation")(inputs)                          # mild augmentation at 32x32
    x = layers.Resizing(VIT_IMG_SIZE, VIT_IMG_SIZE, interpolation="bilinear",
                        dtype="float32", name="resize_to_224")(x)               # 32x32 -> 224x224 on the GPU
    x = layers.Rescaling(scale=vit_scale, offset=vit_offset,
                         dtype="float32", name="vit_normalise")(x)              # same normalisation as pretr
    tokens = backbone(x)                                                        # (batch, 197, 768)
    cls_feature = tokens[:, 0]                                                  # [CLS] token = image represe
    x = layers.Dropout(0.2)(cls_feature)
    outputs = layers.Dense(NUM_CLASSES, activation="softmax",
                           dtype="float32", name="cifar10_head")(x)             # NEW 10-class head
    return keras.Model(inputs, outputs, name="vit_cifar10")

vit_model = build_vit_classifier(vit_backbone)
vit_model.summary()

vit_params = vit_model.count_params()
print(f"\nViT total parameters: {vit_params:,}")
print("Input  shape:", vit_model.input_shape, "| Output shape:", vit_model.output_shape)
assert vit_model.output_shape == (None, NUM_CLASSES)
```

Model: "vit_cifar10"

Layer (type)	Output Shape	Param #
cifar_image (InputLayer)	(None, 32, 32, 3)	0
vit_augmentation (Sequential)	(None, 32, 32, 3)	0
resize_to_224 (Resizing)	(None, 224, 224, 3)	0
vit_normalise (Rescaling)	(None, 224, 224, 3)	0
vi_t_backbone (ViTBackbone)	(None, 197, 768)	85,798,656
get_item (GetItem)	(None, 768)	0
dropout_16 (Dropout)	(None, 768)	0
cifar10_head (Dense)	(None, 10)	7,690

Total params: 85,806,346 (327.33 MB)
Trainable params: 85,806,346 (327.33 MB)
Non-trainable params: 0 (0.00 B)

```
ViT total parameters: 85,806,346
Input  shape: (None, 32, 32, 3) | Output shape: (None, 10)
```

```python
def count_trainable(model):
    return int(sum(np.prod(w.shape) for w in model.trainable_weights))

# ---------------- Stage 1: feature extraction (backbone frozen) ----------------
vit_backbone.trainable = False
vit_model.compile(optimizer=keras.optimizers.Adam(1e-3, clipnorm=1.0),
                  loss="sparse_categorical_crossentropy", metrics=["accuracy"])
print(f"Stage 1 trainable parameters: {count_trainable(vit_model):,} of {vit_params:,} (only the new head)")

stage1_callbacks = [
    keras.callbacks.EarlyStopping(monitor="val_accuracy", patience=2, restore_best_weights=True, verbose=1),
    keras.callbacks.ReduceLROnPlateau(monitor="val_loss", factor=0.5, patience=1, min_lr=1e-5, verbose=1),
]
t0 = time.time()
vit_hist1 = vit_model.fit(vit_train_ds, validation_data=vit_val_ds,
                          epochs=VIT_STAGE1_EPOCHS, callbacks=stage1_callbacks, verbose=2)
vit_time_stage1 = time.time() - t0
print(f"\nStage 1 time: {vit_time_stage1:.1f} s | best val acc: {max(vit_hist1.history['val_accuracy']):.4f}")
```

```
Stage 1 trainable parameters: 7,690 of 85,806,346 (only the new head)
Epoch 1/2
704/704 - 359s - 510ms/step - accuracy: 0.8687 - loss: 0.4064 - val_accuracy: 0.9636 - val_loss: 0.1121 - le
Epoch 2/2
704/704 - 344s - 488ms/step - accuracy: 0.9056 - loss: 0.2824 - val_accuracy: 0.9674 - val_loss: 0.1015 - le
Restoring model weights from the end of the best epoch: 2.

Stage 1 time: 703.0 s | best val acc: 0.9674
```

```python
stage2_callbacks = [
    keras.callbacks.EarlyStopping(monitor="val_accuracy", patience=2, restore_best_weights=True, verbose=1),
    keras.callbacks.ReduceLROnPlateau(monitor="val_loss", factor=0.5, patience=1, min_lr=1e-6, verbose=1),
]
t0 = time.time()
vit_hist2 = vit_model.fit(vit_train_ds, validation_data=vit_val_ds,
                          epochs=VIT_STAGE2_EPOCHS, callbacks=stage2_callbacks, verbose=2)
vit_time_stage2 = time.time() - t0

vit_train_time = vit_time_stage1 + vit_time_stage2
vit_hist = {k: vit_hist1.history[k] + vit_hist2.history[k] for k in vit_hist1.history}   # merge both stage
vit_stage1_epochs_run = len(vit_hist1.history["loss"])
print(f"\nStage 2 time: {vit_time_stage2:.1f} s")
print(f"Total ViT training time : {vit_train_time:.1f} s ({vit_train_time/60:.2f} min)")
print(f"Final training accuracy : {vit_hist['accuracy'][-1]:.4f}")
print(f"Final validation acc.   : {vit_hist['val_accuracy'][-1]:.4f}")
print(f"Best validation acc.    : {max(vit_hist['val_accuracy']):.4f}")
```

```
Stage 2: 4 of 12 transformer blocks unfrozen
Stage 2 trainable parameters: 28,360,714 of 85,806,346
Epoch 1/3
704/704 - 571s - 812ms/step - accuracy: 0.9332 - loss: 0.2033 - val_accuracy: 0.9780 - val_loss: 0.0711 - l
Epoch 2/3
704/704 - 548s - 779ms/step - accuracy: 0.9536 - loss: 0.1419 - val_accuracy: 0.9794 - val_loss: 0.0629 - l
Epoch 3/3
704/704 - 548s - 778ms/step - accuracy: 0.9614 - loss: 0.1123 - val_accuracy: 0.9790 - val_loss: 0.0596 - l
Restoring model weights from the end of the best epoch: 2.

Stage 2 time: 1667.9 s
Total ViT training time : 2370.9 s (39.51 min)
Final training accuracy : 0.9614
Final validation acc.   : 0.9790
Best validation acc.    : 0.9794
```

```python
vit_res = evaluate_model(vit_model, vit_test_ds, y_test)
print(f"ViT Test loss     : {vit_res['loss']:.4f}")
print(f"ViT Test accuracy : {vit_res['accuracy']:.4f}  ({vit_res['accuracy']*100:.2f}%)")
print(f"ViT Test macro-F1 : {vit_res['macro_f1']:.4f}\n")
print("Classification report (ViT):\n")
print(vit_res["report"])
```

```
ViT Test loss     : 0.0700
ViT Test accuracy : 0.9776  (97.76%)
ViT Test macro-F1 : 0.9776

Classification report (ViT):

              precision    recall  f1-score   support

    airplane     0.9859    0.9820    0.9840      1000
  automobile     0.9742    0.9830    0.9786      1000
        bird     0.9839    0.9790    0.9815      1000
         cat     0.9609    0.9350    0.9478      1000
        deer     0.9869    0.9790    0.9829      1000
         dog     0.9402    0.9740    0.9568      1000
        frog     0.9851    0.9910    0.9880      1000
       horse     0.9960    0.9860    0.9910      1000
        ship     0.9890    0.9900    0.9895      1000
       truck     0.9750    0.9770    0.9760      1000

    accuracy                         0.9776     10000
   macro avg     0.9777    0.9776    0.9776     10000
weighted avg     0.9777    0.9776    0.9776     10000
```

Metric	CNN	Pretrained ViT
Test Accuracy	0.8049	0.9776
Test Loss	0.5648	0.0700
Training Time (s)	221.2	2,370.9
Training Time (min)	3.6868	39.5
Number of Parameters	323,946	85,806,346
Best Validation Accuracy	0.8106	0.9794
Test F1 Score (macro)	0.8032	0.9776
Epochs Trained	20.0	5.0000

Table also saved to cnn_vs_vit_comparison.csv (Colab file browser, left sidebar).

